



“Signova: Sign Language Recognition and Translation System”

A Project Report submitted in partial fulfilment of the requirements of the Project Based Learning (PBL) component of

Database Systems Engineering and Distributed Backend Development (25CS1302E)

BACHELOR OF TECHNOLOGY

in

**DEPARTMENT OF Artificial Intelligence and Data Science
AND
CSIT**

By

MAHATHI CHEKKA 2520080030

SARANYA ALURU 2520080012

KIRAN 2520090035

Team 24 | Section 13

Under the Esteemed Guidance of

Dr M Gayatri

K L (Deemed to be) University

KLH Bachupally Campus, Hyderabad

Academic Year 2026–27



K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

The Project Report entitled “Signova: Sign Language Recognition and Translation System” is a record of bonafide work of MAHATHI - 2520080030, SARANYA - 2520080012, KIRAN - 2520090035, submitted as part of the Project Based Learning component of the course Database Systems Engineering and Distributed Backend Development (25CS1302E) for the B.Tech programme in Computer Science and Engineering at K L (Deemed to be) University. The results embodied in this report have not been copied from any other department, university or institute.

MAHATHI – 2520080030

SARANYA – 2520080012

KIRAN – 2520090035



K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project based report entitled “Signova: Sign Language Recognition and Translation System” is a bonafide work done and submitted by MAHATHI - 2520080030, SARANYA - 2520080012, KIRAN - 2520090035 in partial fulfilment of the requirements of the course Database Systems Engineering and Distributed Backend Development (25CS1302E) in the Department of Computer Science and Engineering, K L (Deemed to be) University, during the academic year 2026–27.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr M Gayatri, for her guidance and support throughout the development of the Signova project.

We also thank the faculty and staff of the Department of Computer Science and Engineering, KLH Bachupally Campus, for providing the facilities and academic support required to complete this project.

We are grateful to our classmates, friends and family for their encouragement and assistance during the project development.

MAHATHI – 2520080030

SARANYA – 2520080012

KIRAN – 2520090035

ABSTRACT

Signova is a web-based sign-language recognition prototype designed to recognise selected hand gestures and convert them into readable words. The system uses webcam input, OpenCV for frame processing, MediaPipe for hand landmark detection, and a landmark-based recognition model for sign prediction.

A Flask backend provides REST API communication between the recognition system and the React frontend. MySQL is used for structured storage of sign information and recognition history. The project also supports basic formation and display of recognised words as a sentence.

The current implementation focuses on a limited vocabulary suitable for a student-level prototype. It provides a foundation that can later be extended with more training data, additional signs, dynamic gesture recognition, improved sentence formation and voice output.

INDEX

S. No.	Chapters	Topics	Page No.
		Acknowledgement Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	8–10
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	11–12
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Computer vision and recognition 3.4 Database 3.5 Dataset 3.6 Development environment and version control	13–14
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer 4.3 Data flow 4.4 Deployment architecture	15–16
5	Design	5.1 Design overview 5.2 ER diagram / Database schema	17–18
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic 6.3 API integration 6.4 Database integration	19–20
7	Features	7.1 List of main features 7.2 How each feature works 7.3 Supported signs	21–22
8	Testing	8.1 Backend / API testing 8.2 Integration testing 8.3 Test cases and results 8.4 Dataset / recognition testing	23

9	Deployment	9.1 Steps to deploy (local) 9.2 Environment configuration 9.3 Current deployment scope	24
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations	25
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimisations	26
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	27
13	References	Documentation and dataset sources	28
14	Appendices	Screenshots of the app Installation/setup instructions	29–30

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

Sign language is the primary means of communication for many deaf and hard-of-hearing people. It is a complete visual language in which meaning is carried by the shape, position and movement of the hands, together with facial expression and body posture. In India, Indian Sign Language (ISL) is used across the country, yet very few hearing people learn it. As a result, a signer and a non-signer often cannot hold even a simple conversation without a human interpreter, and interpreters are not available in most everyday situations such as a shop counter, a classroom or a hospital reception.

Advances in computer vision have made it practical to analyse hand gestures with nothing more than an ordinary webcam. Libraries such as OpenCV make it straightforward to capture and process video frames, and MediaPipe provides a pre-trained hand-tracking solution that locates 21 landmark points on each hand in real time. These landmark coordinates describe the pose of the hand in a compact numerical form, which makes them well suited as input to a machine-learning classifier. This removes the need for special hardware such as sensor gloves or depth cameras.

Signova explores this idea as a web-based prototype. A user shows a sign to the webcam, the system detects the hand landmarks, a recognition model predicts which sign from a selected vocabulary is being shown, and the result appears as a readable word in the browser. Recognised words are placed one after another to form a simple sentence, and every recognition is recorded in a database so that the history can be reviewed later. The project brings together computer vision, a REST backend built with Flask, a MySQL relational database and a React frontend.

The project was developed as part of the course Database Systems Engineering and Distributed Backend Development (25CS1302E) under the Project Based Learning (PBL) scheme for the academic year 2026–27 at KLH Bachupally Campus. For this reason the database design and the backend API are treated as first-class parts of the system rather than as an afterthought to the recognition model.

1.2 Problem Statement

There is a communication gap when people who are unfamiliar with sign language interact with sign-language users. The non-signer cannot understand the gestures, and the signer has no convenient way to convert them into text on the spot. Human interpreters solve the problem but are scarce and cannot be present for every interaction.

The problem addressed by this project is therefore: **to build a simple system that recognises a selected set of sign-language gestures through a standard webcam and presents them as text in a web interface, while storing sign information and recognition history in a relational database.** The system is intended to demonstrate how technology can assist this interaction, not to replace interpreters or to translate the full language.

1.3 Scope of the Project

The scope of Signova is deliberately limited so that a complete, working pipeline can be delivered within one semester. The following are within scope:

- ♦ Webcam-based recognition of selected sign-language gestures.
- ♦ Hand landmark extraction using MediaPipe.
- ♦ Landmark-based sign classification using a trained recognition model.
- ♦ A Flask REST API for communication between the recognition system and the user interface.
- ♦ A React-based user interface for live recognition.
- ♦ MySQL storage for sign information and recognition history.
- ♦ Basic sentence display formed from sequentially recognised words.

The following are outside the scope of the initial version:

- ♦ Translation of the complete sign language. The prototype supports a small, selected vocabulary.
- ♦ Understanding of continuous, natural signing. Each sign is recognised individually.
- ♦ Dynamic gestures that depend on motion over time.
- ♦ Grammar-aware sentence construction, voice output and mobile or cloud deployment.

1.4 Objectives of the Project

1. Detect hand landmarks from webcam input.
2. Recognise a selected vocabulary of sign-language gestures.
3. Provide recognition results through a web interface.
4. Store sign information and recognition history using MySQL.
5. Create a foundation for future improvements in vocabulary, accuracy and sentence formation.

Each objective maps to a part of the system. The first two are met by the computer-vision and recognition module, the third by the Flask API and React frontend, and the fourth by the

database layer. The fifth is met by keeping the modules separate, so that any one of them can be improved without redesigning the rest.

1.5 Importance of the Application

The project demonstrates a practical application of computer vision and machine learning for accessibility. Even with a small vocabulary, a working gesture-to-text pipeline shows how a common device such as a laptop with a webcam can help bridge everyday communication between signers and non-signers.

It also has academic value. Signova is an example of integrating a recognition model with a REST backend, a relational database and a frontend interface. The team had to make these independent pieces communicate reliably, which is the central skill in building any distributed application. The Signs page of the application additionally works as a small learning aid, since it lists each supported sign with its meaning.

1.6 Target Users / Audience

- ♦ Students and developers exploring sign-language recognition.
- ♦ Users who want a simple demonstration of gesture-to-text recognition.
- ♦ Academic evaluators interested in computer vision, databases and web application integration.
- ♦ Future developers who may extend the prototype with more signs or features.

CHAPTER 2 – SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Signova is designed to run on ordinary student hardware. No graphics card or special sensor is required, because hand tracking is performed on the CPU and the recognition model works on a small set of landmark coordinates rather than on full images.

Component	Requirement
Computer	Laptop or desktop computer
Camera	Webcam or built-in camera
Memory	Minimum 4 GB RAM recommended for the basic prototype
Input devices	Keyboard and mouse
Network	Internet connection for installing dependencies and downloading dataset resources during development

A camera with reasonable clarity and a well-lit environment improve landmark detection. The internet connection is needed only while setting up the project; once the dependencies are installed the application runs entirely on the local machine.

2.2 Software Requirements

Software	Purpose
Windows or another OS capable of running Python and Node.js	Operating system
Python 3.x	Runs the recognition pipeline and the Flask backend
Node.js and npm	Runs and builds the React frontend
MySQL Server	Stores sign information and recognition history
Modern web browser	Displays the user interface and provides camera access
VS Code or another code editor	Development

2.3 Development Tools and Frameworks

Tool / Framework	Role in the project
React	Browser-based user interface
Python Flask	REST API backend
OpenCV	Webcam frame handling and image processing
MediaPipe	Hand landmark detection
Recognition / model libraries	Training and running the landmark-based sign classifier
MySQL	Relational database
Git / GitHub	Version control and collaboration

The backend dependencies are kept in a Python virtual environment so that the project does not interfere with other Python installations on the machine. The frontend dependencies are managed by npm and recorded in the project's package file.

CHAPTER 3 – TECHNOLOGY STACK

3.1 Front-End

The user interface is built with React. React was chosen because it lets the interface be described as a set of components whose display updates automatically when the underlying state changes. This suits Signova well: the detected sign, the current sentence and the translation history all change every time a new recognition result arrives, and React re-renders only the parts of the page that depend on the changed state.

The interface is organised into four sections reached from a sidebar: Recognition, Signs, Translation History and About. The Recognition page is the main working screen, containing the camera input panel, the detected sign, the current sentence and the most recent history entries. The Signs page lists the vocabulary supported by the system.

3.2 Back-End

The backend is written in Python using Flask. Flask is a lightweight web framework that makes it simple to expose Python functions as REST API endpoints. Python was the natural choice for the backend because OpenCV, MediaPipe and the model libraries are all Python libraries, so the recognition code and the API can live in the same application.

The backend exposes endpoints that allow the React frontend to request a recognition and to retrieve sign-related information. The primary endpoint used by the prototype is `/recognize`. Responses are returned as JSON, which the frontend reads directly.

3.3 Computer Vision and Recognition

OpenCV handles webcam frames and basic image processing. MediaPipe detects hand landmarks: for each detected hand it returns 21 points, each with coordinates describing its position in the frame. The landmark coordinates are converted into a feature vector for the recognition model.

Using landmarks instead of raw images has clear advantages for a student project. The feature vector is small, so training and prediction are fast. The features are also largely independent of skin tone, clothing and background, because only the geometry of the hand is passed to the model.

3.4 Database

MySQL is used as the relational database for structured sign information and recognition history. A relational database is appropriate because the data has a clear, fixed structure and a

natural relationship: every recognition event refers to exactly one known sign. This relationship is enforced with a foreign key, which guarantees that the history can never contain a reference to a sign that does not exist.

3.5 Dataset

A downloaded ISL sign-language dataset, obtained from a Hugging Face dataset repository, was used during development. The images were passed through the landmark-extraction step, which produced **1,752 samples** written to `backend/isl_landmarks.csv`. Each row of this file holds the landmark features for one sample together with its sign label. The project uses a selected vocabulary rather than attempting to cover the complete language.

3.6 Development Environment and Version Control

The project is developed in Visual Studio Code. PowerShell is used for backend commands and npm is used for the React frontend. Git and GitHub are used for version control, so that the three team members can work on the recognition, backend and frontend modules in parallel and merge their work.

CHAPTER 4 – SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

Signova is organised as a pipeline. A video frame enters at the webcam and passes through frame processing, landmark detection and sign prediction. The prediction is then served by the Flask API to the React frontend and recorded in MySQL.

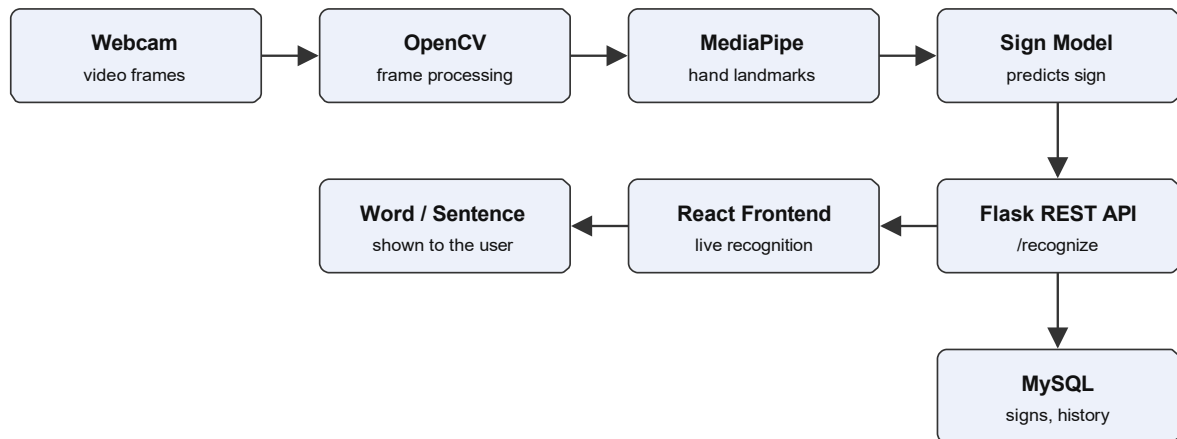


Figure 4.1: High-level architecture of Signova

4.2 Description of Each Layer

Input Layer

The webcam captures the user's hand gesture as video frames. The user starts and stops the camera from the Recognition page.

Computer Vision Layer

OpenCV processes the frames and MediaPipe detects the hand landmarks. If no hand is found in a frame, no landmarks are produced and no prediction is attempted.

Recognition Layer

The landmark features are passed to the trained recognition model, which predicts one sign from the selected vocabulary.

Backend Layer

Flask provides REST endpoints for recognition and sign information. It is the only component that the frontend talks to, and the only component that talks to the database.

Database Layer

MySQL stores structured sign information in the Signs table and recognition events in the Recognition History table.

Frontend Layer

React displays the live recognition result, the recognised words and the basic sentence output, along with the translation history.

4.3 Data Flow

1. Capture the hand gesture through the webcam.
2. Process the frame using OpenCV.
3. Detect hand landmarks using MediaPipe.
4. Prepare landmark features for the recognition model.
5. Predict the sign.
6. Expose the result through Flask.
7. Display the result in React and store the relevant history in MySQL.

4.4 Deployment Architecture

The current system is deployed locally. The Flask backend and the React development server run as two separate processes on the same machine, and the MySQL server runs alongside them. The browser loads the React application, which sends HTTP requests to the local Flask address. There is no cloud hosting or CI/CD pipeline in this version; these are identified as future work in Chapter 11.

CHAPTER 5 – DESIGN

5.1 Design Overview

The design separates recognition, backend communication, database storage and frontend presentation so that individual components can be improved without redesigning the complete application. For example, the recognition model can be retrained with more signs without any change to the frontend, as long as the API continues to return a sign label in the same form.

5.2 ER Diagram / Database Schema

The database contains two entities. **Signs** holds one record for each sign the system knows about. **Recognition History** holds one record for each time a sign was recognised.

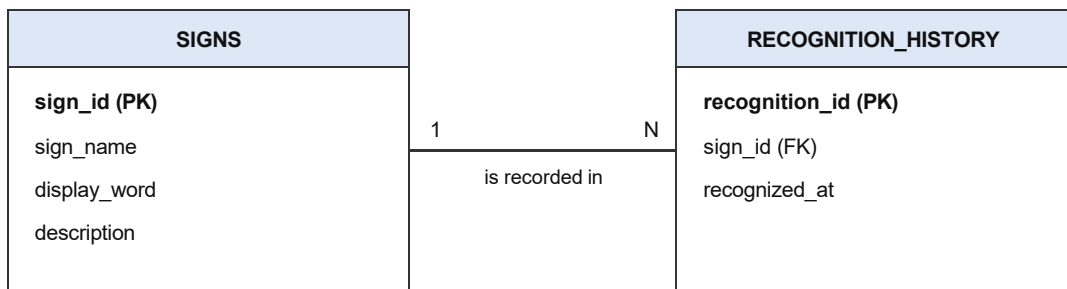


Figure 5.1: ER diagram of the Signova database

Table	Column	Key	Description
SIGNS	sign_id	Primary key	Unique identifier of the sign
	sign_name		Internal label of the sign, as produced by the model
	display_word		Word shown to the user
	description		Short meaning of the sign
RECOGNITION_HISTORY	recognition_id	Primary key	Unique identifier of the recognition event
	sign_id	Foreign key	References SIGNS.sign_id
	recognized_at		Date and time of the recognition

Relationship: one sign can have many recognition-history records. The `sign_id` field in Recognition History references the corresponding record in Signs, giving a one-to-many relationship.

The design is normalised. The descriptive details of a sign are stored once in the Signs table, and the history table stores only the sign's identifier and the time. When the history is displayed, the word is obtained by joining the two tables. This avoids repeating the sign's text in every history row and means that a correction to a sign's display word is automatically reflected throughout the history.

CHAPTER 6 – IMPLEMENTATION

6.1 Module-Wise Implementation

The work was divided into three modules that match the layers of the architecture.

Module 1 – Hand Detection and Sign Recognition

- ♦ The webcam captures live frames.
- ♦ OpenCV handles frame processing.
- ♦ MediaPipe detects hand landmarks.
- ♦ Landmarks are converted into numerical features.
- ♦ The recognition model predicts a selected sign.

The same landmark-extraction step is used in two places. During development it was run over the downloaded dataset to produce the training file `backend/isl_landmarks.csv`. At run time it is applied to the live frame, so the model receives features in exactly the form it was trained on.

Module 2 – Flask Backend and MySQL

- ♦ Flask provides the REST API endpoints.
- ♦ The backend connects the recognition module with the web interface.
- ♦ MySQL stores sign information and recognition history.
- ♦ SQL operations insert and retrieve records.

Module 3 – React Frontend and Sentence Recognition

- ♦ React provides the user interface.
- ♦ The current recognised sign is displayed.
- ♦ Sequentially recognised words are combined for basic sentence display.
- ♦ The recognition history is shown in the interface.

6.2 Front-End Logic

The React interface communicates with Flask, receives recognition results and updates the displayed sign and sentence state. On the Recognition page the Camera Input panel shows the camera status and provides the Start Camera and Recognize Now controls. When a result is returned, three parts of the page update: the Detected Sign card shows the new word, the word is appended to the Current Sentence, and a new entry with its date and time appears at the top of the Translation History. The Clear button empties the current sentence so that a new one can be started.

6.3 API Integration

The Flask backend exposes endpoints for recognition and for sign information. The primary recognition endpoint used by the prototype is `/recognize`. The frontend calls this endpoint, the backend runs the recognition pipeline and returns the predicted sign, and the frontend displays it. The sign-information endpoints supply the vocabulary shown on the Signs page.

6.4 Database Integration

The backend uses MySQL to manage the structured application data. Sign records are stored in Signs, while recognition events are recorded in Recognition History. When a sign is recognised, the backend inserts a new history row containing the sign's identifier and the current time. When the history is requested, the backend selects the rows joined with the Signs table, newest first.

CHAPTER 7 – FEATURES

7.1 List of Main Features

- ♦ Webcam-based hand gesture input.
- ♦ Hand landmark detection.
- ♦ Selected-sign recognition.
- ♦ Real-time recognition API.
- ♦ React-based frontend.
- ♦ MySQL database storage.
- ♦ Recognition history.
- ♦ Basic word-to-sentence display.

7.2 How Each Feature Works

User perspective. The user opens the Signova web interface and activates the webcam from the Recognition page. The user shows a sign to the camera and the recognised word appears in the Detected Sign card. As more signs are shown, the words are displayed one after another in the Current Sentence box. The Translation History lists what was recognised and when. The Signs page lets the user browse the vocabulary the system currently supports, with a short meaning for each sign.

Technical description. A gesture is captured and processed through the computer-vision pipeline: OpenCV reads the frame, MediaPipe extracts the hand landmarks and the features are passed to the model. The model predicts a sign, and the result is sent to the frontend through the Flask API as a JSON response. The frontend updates its state, and the event is stored in the Recognition History table.

7.3 Supported Signs

The application's Signs page currently lists the following eight signs.

Sign	Meaning
Hello	A friendly greeting used when meeting someone.
Yes	Used to express agreement or confirmation.
No	Used to express disagreement or refusal.
Please	Used when making a polite request.
Thank You	Used to express gratitude or appreciation.
I Love You	Used to express love and affection.
Namaste	A traditional Indian greeting.
House	A place where a person lives.

CHAPTER 8 – TESTING

8.1 Backend / API Testing

The Flask backend was tested by running the application locally and sending recognition requests to `/recognize`. A successful request returns an HTTP 200 response when the endpoint processes the input.

8.2 Integration Testing

Integration testing focuses on the connection between the webcam pipeline, the recognition model, the Flask backend, the React frontend and the database. The complete pipeline was run locally from end to end: a sign was shown to the camera, the result was checked in the Detected Sign card, and the corresponding entry was checked in the Translation History.

8.3 Test Cases and Results

Test Case	Input	Expected Result	Status
TC01	Webcam starts	Camera frames available	Pass
TC02	Hand in camera view	Landmarks detected	Pass
TC03	Known selected sign	Sign label returned	Pass
TC04	Recognition API request	HTTP response received	Pass
TC05	Frontend connection	Result displayed	Pass
TC06	Recognition history	Record can be stored/retrieved	Pass

All six test cases passed.

8.4 Dataset / Recognition Testing

Landmark extraction completed with 1,752 samples written to `backend/isl_landmarks.csv`. Recognition quality can vary with hand position, lighting, background, camera angle and the similarity between signs. No formal accuracy figure is claimed for the prototype; measuring accuracy on a held-out test set is listed as future work.

CHAPTER 9 – DEPLOYMENT

9.1 Steps to Deploy

The current project is designed to run locally. The Flask backend and the React frontend are started separately, each in its own terminal.

Backend

```
python app.py
```

Frontend

```
npm run dev
```

Once both are running, the application is opened in the browser at the local address printed by the frontend development server.

9.2 Environment Configuration

- ♦ A Python virtual environment contains the backend dependencies.
- ♦ Node.js/npm manages the frontend dependencies.
- ♦ The MySQL server must be available when database functionality is required.
- ♦ The frontend must use the correct local Flask API address.

9.3 Current Deployment Scope

The current implementation is a local development prototype. Cloud deployment is considered future work.

CHAPTER 10 – CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- ♦ **Variation in hand position and camera angle.** The same sign looks different depending on how far the hand is from the camera and how it is turned.
- ♦ **Similar-looking gestures.** Some signs differ only slightly in finger position and can be difficult to distinguish.
- ♦ **Limited training data.** A small dataset can reduce how well the model generalises to new users.
- ♦ **Real-time processing.** Real-time webcam processing requires efficient frame handling.
- ♦ **Integration.** The frontend, backend and recognition components must communicate consistently.

10.2 Solutions Applied

- ♦ Hand landmarks are used instead of raw images as model features.
- ♦ Landmark-based features are normalised where applicable.
- ♦ A modular Flask API keeps the recognition code separate from the interface.
- ♦ Structured information is stored in MySQL.
- ♦ The complete pipeline is tested locally before integration.

10.3 Current Limitations

- ♦ Limited sign vocabulary.
- ♦ Not a complete translation of continuous sign language.
- ♦ Basic sentence formation: words are placed in the order recognised, without grammar.
- ♦ Accuracy depends on the training data and the capture conditions.
- ♦ Primarily an academic prototype that runs locally.

CHAPTER 11 – FUTURE ENHANCEMENTS

11.1 Planned Features

- ♦ Expand the vocabulary and the training data.
- ♦ Improve the classification of similar signs.
- ♦ Support dynamic gestures using temporal models.
- ♦ Improve sentence formation using natural-language processing.
- ♦ Add text-to-speech output.

11.2 Possible Integrations or Optimisations

- ♦ Develop a mobile application.
- ♦ Deploy to a cloud platform.
- ♦ Improve robustness to lighting, background and camera variations.
- ♦ Measure accuracy on a separate test set and report it for each sign.

CHAPTER 12 – CONCLUSION

12.1 Summary of the Project

Signova demonstrates an end-to-end sign-language recognition prototype using webcam input, computer vision, landmark-based recognition, REST APIs, a relational database and a web frontend. The project is intentionally scoped as a basic academic implementation while providing a structure for future development.

12.2 What Was Achieved

- ♦ Webcam-based hand input and landmark detection.
- ♦ A recognition pipeline for a selected vocabulary.
- ♦ A Flask backend/API layer.
- ♦ MySQL structured data storage.
- ♦ A React frontend interface.
- ♦ A basic sentence-recognition workflow.
- ♦ A dataset processed into 1,752 landmark samples.

12.3 Skills Learned During Development

- ♦ Computer vision and hand landmark processing.
- ♦ Recognition model integration.
- ♦ REST API development with Flask.
- ♦ React frontend development.
- ♦ MySQL database design and SQL.
- ♦ Full-stack integration and debugging.

CHAPTER 13 – REFERENCES

1. MediaPipe Documentation – Hand Landmarker and MediaPipe Tasks.
2. OpenCV Documentation – Open Source Computer Vision Library.
3. Flask Documentation – Python web application framework and REST API development.
4. React Documentation – React library for building user interfaces.
5. MySQL Documentation – Relational database management and SQL reference.
6. Hugging Face – ISL sign-language dataset repository used during dataset processing.

CHAPTER 14 – APPENDICES

Screenshots of the App

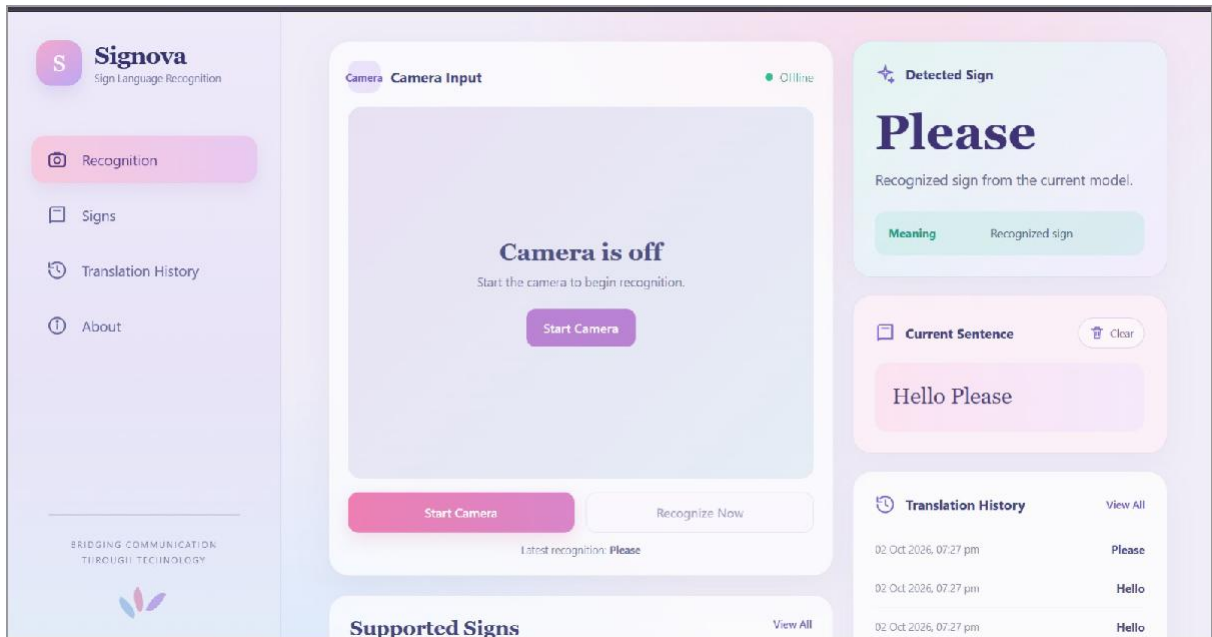


Figure A.1: Recognition page showing camera input, detected sign, current sentence and translation history

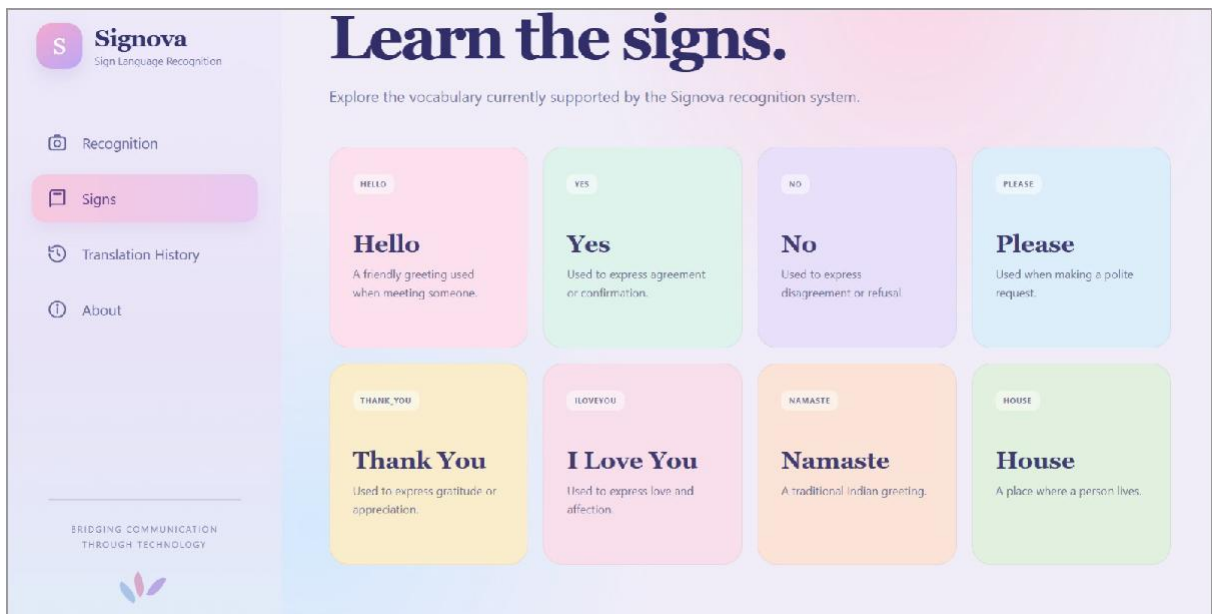


Figure A.2: Signs page listing the supported vocabulary

Installation / Setup Instructions

1. Install Python 3.x, Node.js with npm, and MySQL Server.

2. Create and activate a Python virtual environment in the backend folder and install the backend dependencies.
3. Start the MySQL server and create the Signs and Recognition History tables described in Chapter 5.
4. Start the backend with `python app.py`.
5. In the frontend folder, install the dependencies with `npm install` and start the interface with `npm run dev`.
6. Open the local address shown by the frontend in a browser, allow camera access and choose Start Camera.